

1. What is a computer program?

A computer program is a set of instructions written in a programming language that tells a computer how to perform a specific task. It is executed by the CPU to solve problems or automate processes. Programs can perform calculations, process data, or control hardware. Examples include web browsers, games, and calculators.

2. What is Python?

Python is a high-level, interpreted, and object-oriented programming language developed by Guido van Rossum in 1991. It has a simple syntax, making it easy to learn and use. Python supports multiple programming paradigms such as procedural, object-oriented, and functional programming. It is widely used in web development, data science, AI, and automation.

3. How do you run a Python program?

A Python program can be run in two ways:

- **Interactive Mode:** Type commands directly into the Python interpreter.
 - **Script Mode:** Write code in a .py file and execute it using an IDE like IDLE, VS Code, PyCharm, or by typing `python filename.py` in the command prompt.
-

4. What is the purpose of the `print()` function?

The `print()` function is used to display output on the screen. It can print text, numbers, variables, and expressions. It is mainly used to show results and for debugging purposes.

Example:

```
print("Hello World")
```

5. What are arithmetic operators in Python?

Arithmetic operators perform mathematical operations.

- `+` Addition
- `-` Subtraction
- `*` Multiplication
- `/` Division
- `//` Floor Division
- `%` Modulus

- **** Exponentiation**

Example:

```
a=10  
b=3  
print(a%b)
```

6. What is meant by values and types?

A value is the actual data stored in a variable, such as 10, "Hello", or 3.14. A type defines the kind of value and determines the operations that can be performed on it.

Examples:

- 10 → int
 - 3.5 → float
 - "Python" → str
-

7. How do you check the type of a variable?

Python provides the `type()` function to determine the data type of a variable.

Example:

```
x=10  
print(type(x))
```

Output:

```
<class 'int'>
```

8. What is debugging?

Debugging is the process of identifying, locating, and correcting errors (bugs) in a program. Common types of errors include syntax errors, runtime errors, and logical errors. Debugging tools and print statements help identify problems.

9. What is a variable?

A variable is a named memory location used to store data. Its value can change during program execution. Variables are created automatically when a value is assigned.

Example:

```
age=20
```

10. What rules should be followed while naming variables?

Variable names should:

- Begin with a letter or underscore.
- Not begin with a digit.
- Contain only letters, digits, and underscores.
- Not use Python keywords.
- Be meaningful and case-sensitive.

Example:

```
student_name  
marks1
```

11. What is an assignment statement?

An assignment statement assigns a value to a variable using the = operator.

Example:

```
x=100  
name="Alice"
```

The value on the right side is stored in the variable on the left side.

12. What is the difference between an expression and a statement?

An expression is a combination of values, variables, and operators that produces a result.

Example:

```
5+3
```

A statement is an instruction executed by Python.

Example:

```
x=5+3
```

13. What is script mode in Python?

Script mode allows programs to be written in a file with the .py extension and executed later. It is suitable for writing large programs because the code is saved permanently and can be edited multiple times.

14. What is operator precedence?

Operator precedence determines the order in which operators are evaluated in an expression.

Example:

$2+3*4$

Output:

14

Multiplication is performed before addition.

15. What are string operations?

String operations include:

- Concatenation (+)
 - Repetition (*)
 - Indexing
 - Slicing
 - Comparison
 - Membership (in)
 - String methods such as upper(), lower(), and replace().
-

16. What are comments and why are they used?

Comments are non-executable lines used to explain code. They improve readability and help programmers understand the program.

Single-line:

This is a comment

Multi-line:

"""

This is

a comment

"""

17. What is a function?

A function is a reusable block of code that performs a specific task. It helps reduce code repetition and improves modularity.

Example:

```
def add(a,b):  
    return a+b
```

18. What is the difference between a fruitful and a void function?

A fruitful function returns a value using return.

Example:

```
def square(x):  
    return x*x
```

A void function performs an action but does not return a value.

Example:

```
def greet():  
    print("Hello")
```

19. What are parameters and arguments?

Parameters are variables defined in a function definition.

Arguments are actual values passed to the function during a function call.

Example:

```
def add(a,b): # parameters  
    return a+b
```

```
add(5,10) # arguments
```

20. Why are functions used in programming?

Functions improve code reusability, readability, and modularity. They reduce duplication, simplify debugging, and make large programs easier to maintain. They also allow programs to be divided into smaller manageable parts.

21. What is floor division?

Floor division uses the `//` operator. It returns the largest integer less than or equal to the division result.

Example:

`7//2`

Output:

3

22. What is the modulus operator?

The modulus operator `%` returns the remainder after division.

Example:

`10%3`

Output:

1

It is commonly used to check whether a number is even or odd.

23. What is a Boolean expression?

A Boolean expression evaluates to either True or False. It is commonly used in decision-making and loops.

Example:

`5>3`

Output:

True

24. What are logical operators?

Logical operators combine Boolean expressions.

- `and` – True if both conditions are true.
- `or` – True if at least one condition is true.
- `not` – Reverses the Boolean value.

Example: `a>5 and b<10`

25. What is conditional execution?

Conditional execution allows a program to execute different blocks of code based on a condition. It uses if, elif, and else statements.

Example:

```
if marks >= 35:
    print("Pass")
else:
    print("Fail")
```

26. What is a chained conditional?

A chained conditional is a decision-making structure that uses multiple conditions with if, elif, and else. Python checks each condition in order and executes the first block whose condition is True. If none are true, the else block is executed. It is useful when there are multiple possible outcomes.

Example:

```
marks = 75

if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")
```

27. What is a nested conditional?

A nested conditional is an if statement placed inside another if or else block. It is used when a second condition should be checked only if the first condition is satisfied. Nested conditionals help solve complex decision-making problems.

Example:

```
age = 20
if age >= 18:
    if age >= 21:
        print("Can vote and drink")
    else:
        print("Can vote")
```

28. What is recursion?

Recursion is a technique in which a function calls itself to solve a problem. Each recursive call works on a smaller version of the problem until a stopping condition is reached. It is commonly used for factorial, Fibonacci series, and tree traversal.

Example:

```
def fact(n):  
    if n == 1:  
        return 1  
    return n * fact(n-1)
```

29. What is a base case in recursion?

The base case is the condition that stops recursive calls. Without a base case, the function would continue calling itself indefinitely. Every recursive function must have at least one base case to terminate correctly.

Example:

```
if n == 0:  
    return 1
```

30. What is infinite recursion?

Infinite recursion occurs when a recursive function never reaches its base case. As a result, the function keeps calling itself until Python raises a RecursionError due to exceeding the recursion limit.

Example:

```
def test():  
    test()
```

31. What is keyboard input in Python?

Keyboard input allows users to enter data while a program is running. Python uses the input() function to accept input from the keyboard. The input is returned as a string by default and can be converted using int() or float().

Example:

```
name = input("Enter your name: ")
```

32. What is iteration?

Iteration is the process of executing a block of code repeatedly until a condition becomes false. It is performed using loops such as for and while. Iteration reduces code duplication and automates repetitive tasks.

33. What is variable reassignment?

Variable reassignment means changing the value stored in a variable by assigning it a new value. The previous value is replaced with the new one.

Example:

```
x = 10
```

```
x = 25
```

After reassignment, x contains 25.

34. What is a while loop?

A while loop repeatedly executes a block of code as long as its condition is True. It is useful when the number of iterations is not known in advance.

Example:

```
i = 1
```

```
while i <= 5:
```

```
    print(i)
```

```
    i += 1
```

35. What is the use of break?

The break statement immediately terminates the nearest enclosing loop, even if the loop condition is still true. It is commonly used to stop a loop when a required condition is met.

Example:

```
for i in range(10):
```

```
    if i == 5:
```

```
        break
```

```
    print(i)
```

36. What is a string?

A string is a sequence of characters enclosed in single quotes (' '), double quotes (" "), or triple quotes (''' ''). Strings are used to store and manipulate text data. In Python, strings are immutable.

Example:

```
name = "Python"
```

37. How do you find the length of a string?

The length of a string is determined using the built-in len() function. It returns the total number of characters, including spaces and symbols.

Example:

```
text = "Python"  
print(len(text))
```

Output:

```
6
```

38. What does it mean that strings are immutable?

Immutability means that once a string is created, its contents cannot be changed. Any modification creates a new string instead of changing the existing one.

Example:

```
s = "Hello"  
s = "Hi"
```

39. What is string slicing?

String slicing extracts a portion of a string using index positions. The syntax is string[start:end:step]. The starting index is included, while the ending index is excluded.

Example:

```
s = "Python"  
print(s[1:4])
```

Output:

```
yth
```

40. What is the use of string methods?

String methods perform operations on strings without modifying the original string. Common methods include:

- upper()
- lower()
- replace()
- split()
- strip()
- find()

These methods simplify string manipulation.

41. What is a list?

A list is an ordered, mutable collection of elements enclosed in square brackets ([]). A list can contain elements of different data types and allows duplicate values.

Example:

```
numbers = [10, 20, 30]
```

42. How are lists different from strings?

List	String
Mutable	Immutable
Uses square brackets []	Uses quotes " "
Stores any data type	Stores only characters
Elements can be modified	Characters cannot be modified
Lists are more flexible than strings because they support modification.	

43. What does it mean that lists are mutable?

Mutable means the contents of a list can be changed after creation. Elements can be added, removed, or updated without creating a new list.

Example:

```
numbers = [1, 2, 3]
numbers[1] = 10
```

Output:

```
[1, 10, 3]
```

44. How do you traverse a list?

Traversing a list means visiting each element one by one. It is usually done using a for loop or a while loop.

Example:

```
numbers = [10, 20, 30]
```

```
for num in numbers:
    print(num)
```

45. What are list operations?

Common list operations include:

- Concatenation (+)
- Repetition (*)
- Membership (in)
- Indexing
- Slicing
- Appending
- Inserting
- Removing
- Sorting

These operations make lists powerful for data storage and manipulation.

46. What is list slicing?

List slicing extracts a portion of a list using the syntax:

```
list[start:end]
```

Example:

```
numbers = [10,20,30,40,50]  
print(numbers[1:4])
```

Output: [20, 30, 40]

47. Name some commonly used list methods.

Some commonly used list methods are:

- `append()`
- `extend()`
- `insert()`
- `remove()`
- `pop()`
- `sort()`
- `reverse()`
- `clear()`
- `count()`
- `index()`

These methods help modify and manage list elements efficiently.

48. What is the use of `map()`?

The `map()` function applies a specified function to every element in an iterable and returns a map object. It reduces the need for explicit loops.

Example:

```
numbers = [1,2,3]  
result = list(map(lambda x:x*2, numbers))
```

Output:

[2,4,6]

49. What is the use of filter()?

The filter() function selects elements from an iterable that satisfy a given condition. It returns a filter object.

Example:

```
numbers = [1,2,3,4,5]
result = list(filter(lambda x:x%2==0, numbers))
```

Output:

```
[2,4]
```

50. What is the use of reduce()?

The reduce() function repeatedly applies a function to combine all elements of an iterable into a single value. It is available in the functools module.

Example: from functools import reduce

```
numbers = [1,2,3,4]
result = reduce(lambda x,y:x+y, numbers)
```

Output:

```
10
```

reduce() is commonly used for calculating sums, products, and cumulative results.

51. How do you delete elements from a list?

Elements can be deleted from a list using remove(), pop(), del, or clear().

- remove() deletes a specified value.
- pop() removes an element by index (or the last element by default).
- del deletes an element or slice.
- clear() removes all elements.

Example:

```
numbers = [10, 20, 30, 40]
numbers.remove(20)
numbers.pop()
del numbers[0]
```

52. What is aliasing?

Aliasing occurs when two or more variables refer to the same object in memory. Any modification made through one variable is reflected in the other because both point to the same object.

Example:

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)
```

Output:

```
[1, 2, 3, 4]
```

53. How are lists passed as arguments to functions?

Lists are passed by object reference. The function receives a reference to the original list, so any modifications made inside the function affect the original list unless a copy is used.

Example:

```
def add_item(lst):
    lst.append(100)

numbers = [1, 2]
add_item(numbers)
print(numbers)
```

54. What is a dictionary?

A dictionary is a mutable data structure that stores data as **key–value pairs**. Keys must be unique and immutable, while values can be of any data type. Dictionaries are enclosed in curly braces {}.

Example:

```
student = {
    "name": "Alice",
    "age": 20
}
```

55. How is a dictionary different from a list?

Dictionary	List
Stores key–value pairs	Stores values only
Uses {}	Uses []
Accessed using keys	Accessed using indexes
Keys must be unique	Duplicate elements allowed
Dictionaries are ideal for storing related information.	

56. What is a key–value pair?

A key–value pair is the basic unit of a dictionary. The **key** uniquely identifies the data, while the **value** is the associated information.

Example:

```
student = {  
    "name": "John",  
    "age": 21  
}
```

Here,

- Key → "name"
 - Value → "John"
-

57. How do you loop through a dictionary?

A dictionary can be traversed using methods like:

- keys()
- values()
- items()

Example:

```
student = {"name": "John", "age": 20}
```

```
for key, value in student.items():  
    print(key, value)
```

58. What is a dictionary used as a counter?

A dictionary can count the frequency of items. Each unique item is stored as a key, and its count is stored as the value. This technique is useful for counting words, characters, or numbers.

Example:

```
text = "banana"
count = {}

for ch in text:
    count[ch] = count.get(ch, 0) + 1

print(count)
```

59. What is reverse lookup?

Reverse lookup is the process of finding the key corresponding to a given value in a dictionary. Since dictionaries are optimized for key lookup, reverse lookup usually requires iterating through the dictionary.

Example:

```
student = {"A":90,"B":80}

for key, value in student.items():
    if value == 80:
        print(key)
```

60. What are memos in Python?

A memo is a dictionary used to store previously computed results in recursive functions. This technique, called **memoization**, avoids repeated calculations and improves efficiency.

Example:

Factorial or Fibonacci calculations often use memoization.

61. What is a tuple?

A tuple is an ordered collection of elements enclosed in parentheses (). Tuples can store different data types but are **immutable**, meaning their contents cannot be modified after creation.

Example:

```
student = ("John", 20, "CS")
```

62. How are tuples different from lists?

Tuple	List
Immutable	Mutable
Uses ()	Uses []
Faster	Slightly slower
Suitable for fixed data	Suitable for changing data

63. What is tuple assignment?

Tuple assignment allows multiple variables to be assigned values simultaneously from a tuple.

Example:

```
x, y = (10, 20)
```

```
print(x)
```

```
print(y)
```

It simplifies swapping variables.

64. Can a function return multiple values?

Yes. Python allows functions to return multiple values separated by commas. Internally, the values are returned as a tuple.

Example:

```
def calc(a, b):  
    return a+b, a-b
```

```
sum1, diff = calc(10, 5)
```

65. What are variable-length argument tuples?

Variable-length arguments allow a function to accept any number of positional arguments using `*args`. All arguments are stored in a tuple.

Example:

```
def add(*numbers):  
    print(sum(numbers))  
  
add(1,2,3,4)
```

66. How are lists and tuples related?

Both lists and tuples are ordered collections that support indexing, slicing, iteration, and nesting. A tuple can be converted to a list using `list()`, and a list can be converted to a tuple using `tuple()`.

Example:

```
t = (1,2,3)  
l = list(t)
```

67. How are dictionaries and tuples used together?

The `items()` method returns dictionary entries as tuples. Dictionaries can also use tuples as keys because tuples are immutable.

Example:

```
student = {  
    "John":90,  
    "Alice":95  
}
```

```
for item in student.items():  
    print(item)
```

Output:

```
('John', 90)  
('Alice', 95)
```

68. Why are tuples used instead of lists?

Tuples are preferred when data should not change. They are:

- Immutable
 - Faster than lists
 - Memory efficient
 - Usable as dictionary keys
 - Suitable for fixed collections of data
-

69. What are random numbers in Python?

Random numbers are generated using the random module. They are useful in games, simulations, testing, and password generation.

Example:

```
import random
```

```
print(random.randint(1,10))
```

70. What is a word histogram?

A word histogram is a dictionary that stores each unique word as a key and its frequency as the value. It is commonly used in text analysis and word-count applications.

Example:

Text:

Python is easy.

Python is powerful.

Histogram:

Python → 2

is → 2

easy → 1

powerful → 1

71. What is meant by most common words?

Most common words are the words with the highest frequency in a text. They are identified using a word histogram or frequency count. This technique is used in natural language processing and text mining.

72. What are optional parameters?

Optional parameters are function parameters that have default values. If the caller does not provide an argument, the default value is used.

Example:

```
def greet(name="Guest"):
    print("Hello", name)
```

```
greet()
greet("Alice")
```

73. What is dictionary subtraction?

Dictionary subtraction refers to removing keys or comparing dictionaries to determine which keys exist in one dictionary but not another. Python does not provide a direct subtraction operator for dictionaries, so it is performed using loops or set operations.

Example:

```
d1 = {"a":1,"b":2}
d2 = {"b":2}
```

```
result = {k:v for k,v in d1.items() if k not in d2}
```

74. What is Markov analysis?

Markov analysis is a statistical technique that predicts the next item based only on the current state. In Python, Markov models are used for text generation, language modeling, and predictive analysis by analyzing word transitions.

Applications:

- Chatbots
 - Text prediction
 - Sentence generation
-

75. What is file handling?

File handling is the process of creating, opening, reading, writing, appending, and closing files. Python provides built-in functions such as `open()` and `close()` to perform file operations.

Example:

```
file = open("data.txt", "r")
print(file.read())
file.close()
```

Using the `with` statement is recommended because it automatically closes the file after use.

76. How do you read a file in Python?

A file is read using the `open()` function with **read mode ("r")**. The contents can be read using methods such as `read()`, `readline()`, or `readlines()`. It is recommended to use the `with` statement because it automatically closes the file after reading.

Example:

```
with open("data.txt", "r") as file:
    content = file.read()
    print(content)
```

77. How do you write to a file?

A file is written using the `open()` function in **write mode ("w")** or **append mode ("a")**. The `write()` method writes data to the file. If the file does not exist, Python creates it automatically.

Example:

```
with open("data.txt", "w") as file:
    file.write("Hello Python")
```

78. What is exception handling in files?

Exception handling prevents a program from crashing when file-related errors occur, such as a missing file or permission issues. It uses `try`, `except`, and `finally` blocks to handle errors gracefully.

Example:

```
try:
    file = open("sample.txt", "r")
    print(file.read())
except FileNotFoundError:
    print("File not found.")
```

finally:

```
print("Program completed.")
```

79. What is pickling?

Pickling is the process of converting a Python object into a byte stream so that it can be stored in a file or transmitted over a network. The pickle module is used for serialization, and unpickle restores the original object.

Example:

```
import pickle
```

```
data = [10, 20, 30]
```

```
with open("data.pkl", "wb") as file:
```

```
    pickle.dump(data, file)
```

80. What is the format operator?

The format operator is used to insert values into strings in a readable format. Python supports %, str.format(), and **f-strings**, with f-strings being the recommended approach.

Example:

```
name = "Alice"
```

```
age = 20
```

```
print(f"{name} is {age} years old.")
```

81. What is a class?

A class is a blueprint or template used to create objects. It defines the attributes (data) and methods (functions) that objects of that class will have. Classes support object-oriented programming by grouping related data and behavior.

Example:

```
class Student:
```

```
    pass
```

82. What is an object?

An object is an instance of a class. It contains its own data and can access the methods defined in its class. Multiple objects can be created from the same class, each with different attribute values.

Example:

```
class Student:  
    pass  
  
s1 = Student()
```

83. What are attributes?

Attributes are variables that belong to an object or a class. They store the state or properties of an object. Attributes are accessed using the dot (.) operator.

Example:

```
class Student:  
    def __init__(self, name):  
        self.name = name  
  
s = Student("John")  
print(s.name)
```

84. What is an instance?

An instance is an individual object created from a class. Each instance has its own set of attribute values while sharing the same methods defined by the class.

Example:

```
class Car:  
    pass  
  
c1 = Car()  
c2 = Car()
```

Here, c1 and c2 are separate instances.

85. Are objects mutable in Python?

Some objects in Python are mutable, while others are immutable.

- **Mutable:** Lists, dictionaries, sets, most user-defined objects.

- **Immutable:** Integers, floats, strings, tuples.

Mutable objects can be modified after creation, whereas immutable objects cannot.

86. What is copying an object?

Copying creates a duplicate of an object. Python provides:

- **Shallow copy (`copy.copy()`)** – copies only the outer object.
- **Deep copy (`copy.deepcopy()`)** – copies the object and all nested objects.

Example:

```
import copy

a = [1, 2, [3, 4]]
b = copy.deepcopy(a)
```

87. What is a method?

A method is a function defined inside a class that operates on objects of that class. Methods are called using the dot operator and usually take `self` as the first parameter.

Example:

```
class Student:
    def greet(self):
        print("Hello")

s = Student()
s.greet()
```

88. What is the purpose of the `__init__()` method?

The `__init__()` method is a special constructor that is automatically called when an object is created. It initializes the object's attributes with initial values.

Example:

```
class Student:
    def __init__(self, name):
        self.name = name
```

89. What is the `__str__()` method?

The `__str__()` method defines the string representation of an object. It is automatically called when the object is printed using `print()`.

Example:

```
class Student:
    def __str__(self):
        return "Student Object"

s = Student()
print(s)
```

90. What is operator overloading?

Operator overloading allows existing operators such as `+`, `-`, `*`, and `==` to perform custom operations on user-defined objects. This is achieved by defining special methods like `__add__()` and `__eq__()`.

Example:

```
class Number:
    def __init__(self, value):
        self.value = value

    def __add__(self, other):
        return Number(self.value + other.value)
```

91. What is polymorphism?

Polymorphism means "**many forms**." It allows the same method or function to behave differently for different objects. This improves flexibility and code reusability.

Example:

```
class Dog:
    def sound(self):
        print("Bark")

class Cat:
    def sound(self):
        print("Meow")
```

Both classes have the same method name but different behavior.

92. What is inheritance?

Inheritance is an object-oriented programming feature in which one class acquires the properties and methods of another class. It promotes code reuse and establishes an "is-a" relationship between classes.

Example:

```
class Animal:  
    pass  
  
class Dog(Animal):  
    pass
```

93. What is a base class?

A base class (or parent class) is the class whose properties and methods are inherited by another class. It contains common functionality that can be reused by derived classes.

Example:

```
class Animal:  
    pass
```

Here, Animal is the base class.

94. What is a derived class?

A derived class (or child class) inherits the properties and methods of a base class. It can also add new features or override existing methods.

Example:

```
class Animal:  
    pass  
  
class Dog(Animal):  
    pass
```

Dog is the derived class.

95. What are class attributes?

Class attributes are variables shared by all instances of a class. They are defined inside the class but outside any method. If one object changes a class attribute, the change is reflected for all instances unless overridden.

Example:

```
class Student:  
    college = "VVCE"
```

96. What is data encapsulation?

Data encapsulation is the process of combining data and methods into a single unit (class) while restricting direct access to the object's internal data. It improves data security and maintains data integrity by using access modifiers and methods.

97. What is a class diagram?

A class diagram is a **UML (Unified Modeling Language)** diagram that represents the structure of a program. It shows:

- Classes
- Attributes
- Methods
- Relationships such as inheritance and association

It is used during software design and development.

98. What is type-based dispatch?

Type-based dispatch is the process of selecting the appropriate method or operation based on the type of an object at runtime. It enables polymorphism by allowing the same function call to produce different behaviors for different object types.

99. How is inheritance useful in real applications?

Inheritance improves software development by promoting code reuse and reducing duplication. Common features are written once in the base class and inherited by derived classes, making programs easier to maintain and extend.

Applications:

- Banking systems
 - Student management systems
 - Vehicle management systems
 - Game development
 - GUI frameworks
-

100. Give a real-world example of inheritance.

A common real-world example is a **Vehicle** hierarchy.

- **Base class:** Vehicle
 - Common properties: speed, color, fuel type
 - Common methods: start(), stop()
- **Derived classes:** Car, Bike, and Bus
 - Each inherits the common properties and methods from Vehicle.
 - Each can also define its own unique features, such as:
 - Car → air_conditioning()
 - Bike → kick_start()
 - Bus → passenger_capacity()

Example:

```
class Vehicle:
    def start(self):
        print("Vehicle Started")
```

```
class Car(Vehicle):
    def drive(self):
        print("Car is moving")
```

```
c = Car()
c.start()
c.drive()
```

This demonstrates how inheritance allows a derived class (Car) to reuse the functionality of the base class (Vehicle) while adding its own features.